Clase 3 - Fundamentos: objetos, tipos y errores

Joselina Davyt-Colo

Clase 3: Fundamentos de programación para datos

Subtítulo: Qué es cada cosa antes de escribir código
Institución: Facultad de Ciencias Empresariales y Economía — Universidad de Montevideo

Por qué esta clase

En la clase 2 escribimos un diccionario y lo convertimos en un DataFrame. Funcionó porque copiamos el código.

Esta clase responde una pregunta anterior: **¿qué es cada uno de esos objetos?**

La mayoría de los bloqueos en la primera práctica no son de sintaxis. Son de tipo: el alumno cree que tiene un número y tiene un texto, o cree que tiene una tabla y tiene una lista.

Punto 1. Ejecutar y leer el error

Hay tres maneras de ejecutar código y conviene distinguirlas:

- **Línea por línea** en la consola: para probar.
- **Archivo completo** (`.py`, `.R`): de arriba hacia abajo, siempre en orden.
- **Celda por celda** (notebook, `.qmd`): en el orden en que uno las corre, que no siempre es de arriba hacia abajo.

Nota

La tercera es la que genera más confusión: si se ejecuta la celda 5 antes que la 2, el programa no sabe nada de lo que había en la 2.

El error no es un castigo, es información

```
NameError: name 'df' is not defined
```

Se lee de abajo hacia arriba. La última línea dice **qué** pasó; las de arriba dicen **dónde**.

Este mensaje no significa que el código esté mal escrito. Significa que el nombre `df` nunca fue creado: probablemente la celda que lo creaba no se ejecutó.

Los tres errores del primer día

Python	R	Qué significa
`NameError: name 'x' is not defined`	`Error: object 'x' not found`	El nombre no existe todavía

Python	R	Qué significa
`TypeError`	`Error in ... : non-numeric argument`	La operación no corresponde a ese tipo
`FileNotFoundError`	`cannot open file`	El archivo no está donde se lo buscó

El primero es de orden de ejecución. El segundo es de tipo. El tercero es de ubicación.

Ninguno de los tres se resuelve reescribiendo la línea al azar.

Punto 2. Asignación: el = no es una igualdad

```python
# Python
edad = 35
```

```r
# R
edad <- 35
```

Se lee: *"guardá el valor 35 bajo el nombre `edad`"*.

No es una ecuación. `edad = edad + 1` es perfectamente válido y significa "tomá lo que hay en `edad`, sumale 1 y volvé a guardarlo ahí".

Nombre y valor son cosas distintas

```python
peso = 45.0        # el nombre 'peso' apunta al valor 45.0
peso = 120.5       # ahora apunta a otro valor; el 45.0 se perdió
```

Una consecuencia práctica: si se ejecuta dos veces una celda que dice `df = df.dropna()`, la segunda vez trabaja sobre el resultado de la primera. El nombre es el mismo, el contenido no.

Punto 3. Tipos elementales

Concepto	Python	R	Ejemplo
Entero	`int`	`integer`	`8`
Decimal	`float`	`numeric`/`double`	`45.0`
Texto	`str`	`character`	`"suave"`
Lógico	`bool`	`logical`	`True` / `TRUE`
Fecha	`datetime.date`	`Date`	`2025-03-12`
Categórica	`category` (pandas)	`factor`	`"blando"` de tres valores posibles
Ausente	`None` / `NaN`	`NA`	celda vacía

Los dos últimos no son simétricos

`factor` es un tipo de R base: el lenguaje fue diseñado para datos y ya sabe qué es una variable categórica. En Python no hay equivalente en el lenguaje: la categórica la aporta pandas. Lo mismo con `NA`, que en R es un valor del lenguaje y en Python son dos cosas distintas según el contexto.

Las comillas cambian el tipo

```
peso = 45.0        # float: se le puede calcular un promedio
peso = "45.0"      # str: es la cadena de caracteres "45.0"
```

```
"45.0" + "1"       # devuelve '45.01'  (concatena)
45.0 + 1           # devuelve 46.0     (suma)
```

Este es exactamente el error del práctico: un CSV mal leído deja los pesos como texto, y las operaciones siguen funcionando pero dan resultados sin sentido.

Punto 4. El vector en R y la lista en Python

Acá los dos lenguajes se separan, y la diferencia importa.

```
# R
c(1, 2, 3)
# [1] 1 2 3
```

`c()` combina valores en un **vector**: una secuencia ordenada de elementos **del mismo tipo**.

En R no existen los números sueltos

```
x <- 1
length(x)     # [1] 1
is.vector(x)  # [1] TRUE
```

`1` **es** un vector numérico de longitud 1. R no tiene un tipo "escalar" separado.

Por eso las operaciones se aplican a todo el vector de una vez:

```
c(1, 2, 3) * 10
# [1] 10 20 30
```

No hace falta un bucle. Esto se llama *vectorización* y es el rasgo central de R.

El vector de R homogeneiza los tipos

```
c(1, "a", TRUE)
# [1] "1"    "a"    "TRUE"
```

No da error: convierte **todo** a texto, porque es el único tipo que puede contener a los tres.

Advertencia

Esta conversión es silenciosa. Un solo valor de texto en una columna que debería ser numérica convierte la columna entera en texto.

En Python: lista, y recién después array

```python
[1, 2, 3]             # lista: admite tipos mezclados
[1, "a", True]        # válido, no convierte nada
```

```python
[1, 2, 3] * 10        # NO multiplica: repite la lista 10 veces
```

Para el comportamiento vectorizado de R hay que usar otra estructura:

```python
import numpy as np
np.array([1, 2, 3]) * 10
# array([10, 20, 30])
```

Resumen del punto 4

	R	Python
Estructura básica	vector (`c()`)	lista (`[ ]`)
¿Tipos mezclados?	No: convierte todo	Sí: los conserva
¿Un valor suelto?	Es un vector de largo 1	Es un escalar
Vectorización	Por defecto	Con `numpy`
Primer elemento	`x[1]`	`x[0]`

Punto 5. Diccionario y data frame

Un **diccionario** en Python asocia nombres a valores:

```python
{"textura": ["suave", "rugoso"], "peso": [45.0, 120.5]}
```

Cada clave es el nombre de una columna; cada valor es la columna entera.

Un **data frame** es exactamente eso, con la exigencia de que todas las columnas tengan el mismo largo.

El DataFrame es una colección de columnas

```python
import pandas as pd
df = pd.DataFrame({"textura": ["suave", "rugoso"],
                   "peso": [45.0, 120.5]})
```

```r
df <- data.frame(textura = c("suave", "rugoso"),
                 peso    = c(45.0, 120.5))
```

Cada columna tiene **su propio tipo**. Una fila, en cambio, mezcla tipos: por eso se trabaja por columnas y no por filas.

Punto 6. Función y método

Una **función** se aplica a un objeto:

```python
len(x)          # Python
length(x)       # R
```

Un **método** pertenece al objeto y se invoca con punto:

```python
df.head()       # Python
```

R casi no usa métodos; Python los usa todo el tiempo. Por eso en Python el punto aparece por todos lados.

Los argumentos con nombre son instrucciones

```python
pd.read_csv("datos.csv", sep=";", decimal=",", encoding="latin-1")
```

```r
read.csv("datos.csv", sep = ";", dec = ",", fileEncoding = "latin1")
```

Solo el primer argumento es un **dato** (qué archivo leer). Los otros tres son **instrucciones sobre cómo interpretarlo**.

Importante

Cambiar `sep` no cambia el archivo: cambia la lectura. El mismo archivo leído con dos separadores distintos produce dos tablas distintas.

Punto 7. La comparación devuelve muchos valores

```python
df["peso"] > 50
```

No devuelve un `True`. Devuelve **un `True` o `False` por cada fila**:

```
0    False
1     True
2    False
```

Esa secuencia se llama **máscara booleana** y sirve para dos cosas:

```python
df[df["peso"] > 50]           # filtrar: quedarse con las filas True
(df["peso"] > 50).sum()       # contar: True vale 1, False vale 0
```

Punto 8. Dónde está el archivo

Todo programa se ejecuta **desde algún lugar**: el directorio de trabajo.

```python
from pathlib import Path
Path.cwd()
```

```r
getwd()
```

Una ruta **relativa** (`datos/caja_negra.csv`) se busca a partir de ahí. Una ruta **absoluta** (`C:/Users/jodavyt/...`) contiene el nombre de usuario de quien la escribió y falla en cualquier otra máquina.

Ejercicio: opción múltiple

Una sola respuesta correcta por pregunta. Justificar la elección en la bitácora cuando la consigna lo pida.

Preguntas 1 a 4

1. En R, `c(1, 2, 3)` es:

a. Una función que suma 1 + 2 + 3
b. Un vector numérico de longitud 3
c. Tres objetos independientes guardados juntos
d. Una matriz de tres filas

2. En R, ¿qué es el valor `1` escrito solo?

a. Un número, que no es lo mismo que un vector
b. Un vector numérico de longitud 1
c. Un entero, incompatible con los decimales
d. Depende de si fue asignado a un nombre

3. `c(1, "a", TRUE)` en R devuelve:

a. Un error por mezclar tipos
b. Un vector con un número, un texto y un lógico
c. Un vector de texto: `"1" "a" "TRUE"`
d. Un vector lógico de longitud 3

4. En Python, `[1, 2, 3] * 2` devuelve:

a. `[2, 4, 6]`
b. `[1, 2, 3, 1, 2, 3]`
c. Un error de tipo
d. `[1, 2, 3, 2]`

Preguntas 5 a 8

5. `edad = "35"` guarda:

a. El número 35
b. El texto `"35"`
c. El número 35, con las comillas ignoradas
d. Un error, porque falta declarar el tipo

6. `NameError: name 'df' is not defined` significa que:

a. El archivo `df.csv` no existe
b. El nombre `df` nunca fue creado en esta sesión
c. `df` tiene el tipo equivocado

d. Falta importar pandas

7. En `pd.read_csv("datos.csv", sep=";")`, el `sep=";"` es:

a. Una segunda columna que se quiere leer
b. Una instrucción sobre cómo interpretar el archivo
c. El carácter que se va a eliminar del archivo
d. Un comentario para el lector del código

8. Un mismo archivo leído con `sep=","` y con `sep=";"` produce:

a. Siempre la misma tabla
b. Dos tablas distintas: la lectura es una decisión
c. Un error en uno de los dos casos, siempre
d. La misma tabla, con las columnas en otro orden

Preguntas 9 a 12

9. `df["peso"] > 50` devuelve:

a. La cantidad de pesos mayores a 50
b. Los valores de peso mayores a 50
c. Un `True` o `False` por cada fila
d. Un único `True` si algún peso supera 50

10. La diferencia entre `len(x)` y `x.upper()` es que:

a. La primera es una función aplicada al objeto; la segunda, un método del objeto
b. Ninguna: son dos formas de escribir lo mismo
c. La segunda solo funciona en R
d. La primera es más rápida

11. Un compañero comparte un script que empieza con `pd.read_csv("C:/Users/jodavyt/datos.csv")`. En tu máquina:

a. Funciona igual, porque la ruta está completa
b. Falla, porque esa carpeta no existe en tu computadora
c. Funciona si tenés la misma versión de Windows
d. Funciona si copiás el archivo a tu escritorio

12. En un data frame, ¿qué comparte el mismo tipo de dato?

a. Cada fila
b. Cada columna
c. Toda la tabla
d. Ninguno: cada celda es independiente

Clave de corrección

	Correcta	El error que detecta
1	b	—
2	**b**	Creer que existe un escalar en R
3	**c**	Esperar un error donde hay conversión silenciosa
4	**b**	Suponer que la lista de Python vectoriza como R
5	b	No ver que las comillas definen el tipo
6	b	Confundir nombre inexistente con archivo faltante
7	b	Leer el argumento como dato y no como instrucción
8	b	Creer que el archivo tiene una única lectura posible
9	c	Esperar un valor único donde hay una secuencia
10	a	—
11	b	No distinguir ruta absoluta de relativa
12	b	Pensar la tabla por filas y no por columnas

Las cuatro en negrita son las que suelen fallar más; conviene discutirlas en clase antes de seguir.

Lo que sigue

Con estos ocho puntos ya es posible hacer el práctico de creación y carga de datasets:

- Escribir un diccionario y convertirlo en DataFrame
- Guardarlo en un archivo y volver a leerlo
- Diagnosticar un archivo mal formado
- Distinguir un faltante de un cero